Hamburg University of Technology
Institute of Mathematics
Chair of Numerical Mathematics

Prof. Dr. Sabine Le Borne
M. Sc. Jonas Grams

Hierarchical Algorithms
Winter semester 2024/2025
Exercise sheet 12, Solution hints
Discussed on January 29th, 2025

Homework:

Exercise 1: Let $I = \{1, \dots, 8\}$ and let T_I be the cluster tree depicted in figure 1. From this a block cluster tree $T_{I\times I}$ was constructed whose structure leads to an $\mathcal{H}_p$-matrix with $p = 3$. Further let $\{\mathcal{W}_\sigma\}_{\sigma\in T_I}$ be subspaces which fulfill the nesting principle. Further let $\dim(\mathcal{W}_\sigma) = 1$ for all $\sigma \in T_I$. Let $A \in \mathcal{H}^2(L(T_{I\times I}), \{\mathcal{W}_\sigma\}_{\sigma\in T_I}, \{\mathcal{W}_\sigma\}_{\sigma\in T_I})$ where the admissible leaves are $\mathcal{R}(1)$ matrices. Then for all $\sigma \in L(T_I)$, $\{W_\sigma\} = \{1\}$ is an orthonormal basis of $\mathcal{W}_\sigma = \mathbb{R}$. To compute Ax with $x \in \mathbb{R}^8$, a forward transformation has to be carried out first.

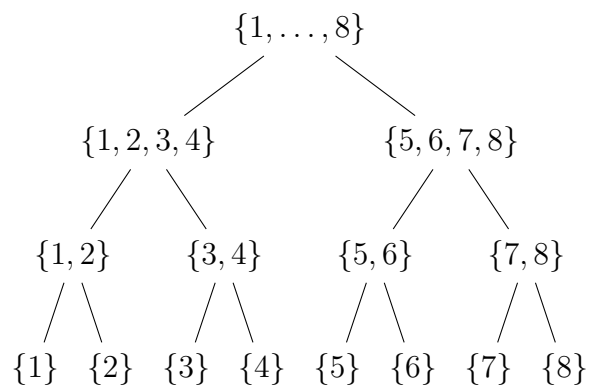

Figure 1: Cluster tree T_I

a) Carry out the forward transformation manually, i. e. represent the transformed vector $\hat{x}_\sigma = W_\sigma^T x_\sigma$ for all $\sigma \in T_I \setminus \{I\}$ only using (basis matrices of the leaves,) transfer matrices and already represented transformed vectors $\hat{x}_{\sigma'}$ with $\sigma' \in \mathrm{S}(\sigma)$.

b) Determine (depending on p) the complexity of this forward transformation (i. e. the computation of $\hat{x}_\sigma$ for all $\sigma \in T_I \setminus \{I\}$). Then show that the complexity of this forward transformation is $\mathcal{O}(n)$ (with $n := 2^p = \#I$).

Solution hints:

a) For all $i \in I$, $W_{\{i\}} = (1) \in \mathbb{R}^{1\times 1}$ is given explicitly. In the following, all transfer matrices are in $\mathbb{R}^{1\times 1}$ because $\dim(\mathcal{W}_\sigma) = 1$ holds for all $\sigma \in T_I$. Therefore one obtains

$$W_{\{i,i+1\}} = \begin{pmatrix} W_{\{i\}} T_{\{i,i+1\}\leftarrow\{i\}} \\ W_{\{i+1\}} T_{\{i,i+1\}\leftarrow\{i+1\}} \end{pmatrix} = \begin{pmatrix} T_{\{i,i+1\}\leftarrow\{i\}} \\ T_{\{i,i+1\}\leftarrow\{i+1\}} \end{pmatrix} \in \mathbb{R}^{2\times 1}$$

$$\{1,\dots,8\}$$

$$b_{1,2,3,4} = \begin{pmatrix} b_{1,2}T_{\{1,2,3,4\}\leftarrow\{1,2\}} \\ b_{3,4}T_{\{1,2,3,4\}\leftarrow\{3,4\}} \end{pmatrix} \qquad b_{5,6,7,8} = \begin{pmatrix} b_{5,6}T_{\{5,6,7,8\}\leftarrow\{5,6\}} \\ b_{7,8}T_{\{5,6,7,8\}\leftarrow\{7,8\}} \end{pmatrix}$$

$$b_{1,2} = \begin{pmatrix} 1T_{\{1,2\}\leftarrow\{1\}} \\ 1T_{\{1,2\}\leftarrow\{2\}} \end{pmatrix} \quad b_{3,4} = \begin{pmatrix} 1T_{\{3,4\}\leftarrow\{3\}} \\ 1T_{\{3,4\}\leftarrow\{4\}} \end{pmatrix} \quad b_{5,6} = \begin{pmatrix} 1T_{\{5,6\}\leftarrow\{5\}} \\ 1T_{\{5,6\}\leftarrow\{6\}} \end{pmatrix} \quad b_{7,8} = \begin{pmatrix} 1T_{\{7,8\}\leftarrow\{7\}} \\ 1T_{\{7,8\}\leftarrow\{8\}} \end{pmatrix}$$

$$b_1 = \begin{pmatrix} 1 \end{pmatrix} \quad b_2 = \begin{pmatrix} 1 \end{pmatrix} \quad b_3 = \begin{pmatrix} 1 \end{pmatrix} \quad b_4 = \begin{pmatrix} 1 \end{pmatrix} \quad b_5 = \begin{pmatrix} 1 \end{pmatrix} \quad b_6 = \begin{pmatrix} 1 \end{pmatrix} \quad b_7 = \begin{pmatrix} 1 \end{pmatrix} \quad b_8 = \begin{pmatrix} 1 \end{pmatrix}$$

Figure 2: Nested basis for T_I with transfer matrices $T_{\{\dots\}\leftarrow\{\dots\}} \in \mathbb{R}^{1\times 1}$.

for all $i \in \{1,3,5,7\}$. For $i \in \{1,5\}$, there holds

$$W_{\{i,i+1,i+2,i+3\}} = \begin{pmatrix} W_{\{i,i+1\}}T_{\{i,i+1,i+2,i+3\}\leftarrow\{i,i+1\}} \\ W_{\{i+2,i+3\}}T_{\{i,i+1,i+2,i+3\}\leftarrow\{i+2,i+3\}} \end{pmatrix} \in \mathbb{R}^{4\times 1}.$$

With that we obtain the following transformed vectors.
For $i \in I$ holds $\hat{x}_{\{i\}} = W_{\{i\}}^T x_i = x_i$. For $i \in \{1,3,5,7\}$ we obtain

$$\hat{x}_{\{i,i+1\}} = W_{\{i,i+1\}}^T \begin{pmatrix} x_i \\ x_{i+1} \end{pmatrix} = T_{\{i,i+1\}\leftarrow\{i\}}^T x_i + T_{\{i,i+1\}\leftarrow\{i+1\}}^T x_{i+1} \in \mathbb{R}$$

and for $i \in \{1,5\}$ it follows that

$$\begin{aligned}
\hat{x}_{\{i,i+1,i+2,i+3\}} &= W_{\{i,i+1,i+2,i+3\}}^T \begin{pmatrix} x_i \\ x_{i+1} \\ x_{i+2} \\ x_{i+3} \end{pmatrix} \\
&= T_{\{i,i+1,i+2,i+3\}\leftarrow\{i,i+1\}}^T W_{\{i,i+1\}}^T \begin{pmatrix} x_i \\ x_{i+1} \end{pmatrix} \\
&\qquad + T_{\{i,i+1,i+2,i+3\}\leftarrow\{i+2,i+3\}}^T W_{\{i+2,i+3\}}^T \begin{pmatrix} x_{i+2} \\ x_{i+3} \end{pmatrix} \\
&= T_{\{i,i+1,i+2,i+3\}\leftarrow\{i,i+1\}}^T \hat{x}_{\{i,i+1\}} + T_{\{i,i+1,i+2,i+3\}\leftarrow\{i+2,i+3\}}^T \hat{x}_{\{i+2,i+3\}} \in \mathbb{R}.
\end{aligned}$$

b) With the representation from part a) it follows that for the computation of $\hat{x}_\sigma$

- 0 operations are necessary, if $\#\sigma = 1$ holds,
- 3 operations are necessary, if $\#\sigma = 2$ holds and
- 9 operations are necessary, if $\#\sigma = 4$ holds (3 computations of $\hat{x}_{\{i,i+1\}}$, $\hat{x}_{\{i+2,i+3\}}$, two multiplications and one addition).

In general, for all $\sigma \in (T_I \setminus \{I\}) \setminus L(T_I)$, the costs for computing $\hat{x}_\sigma \in \mathbb{R}$ are given recursively by

$$3 + 2 \cdot \text{costs for computation of } \hat{x}_{\mathrm{S}(\sigma)} \in \mathbb{R}$$

(this follows from the structure of an $\mathcal{H}_p$-matrix). For $\sigma \in L(T_I)$ there are no costs for the computation of $\hat{x}_\sigma$. In the case $p = 3$, $2 \cdot 9 = 18$ operations are needed to compute $\hat{x}_\sigma$ for each $\sigma \in T_I \setminus \{I\}$.

The costs $\kappa(\hat{\sigma}_\ell)$ for the computation of $\hat{x}_\sigma$ for a σ with $\text{level}(\sigma) = \ell > 0$ are then given by

$$\begin{aligned}
\kappa(\hat{\sigma}_\ell) &= 3 + 2\kappa(\hat{\sigma}_{\ell-1}) \\
&= 3 + 2\left(3 + 2\kappa(\hat{\sigma}_{\ell-2})\right) = 3(1+2) + 2^2\kappa(\hat{\sigma}_{\ell-2}) \\
&= \dots \\
&= 3(1 + 2 + \ldots + 2^{\ell-1}) + 2^\ell \kappa(\hat{\sigma}_0) \\
&= 3(2^\ell - 1).
\end{aligned}$$

Overall the costs of the forward transformation (i. e. the computation of the two $\hat{x}_\sigma$ with $\text{level}(\sigma) = p - 1$, which include the computation of all $\hat{x}_\sigma$ with $\text{level}(\sigma) < p - 1$) depend on p for $p \geq 1$ and are given by

$$2 \cdot 3(2^{p-1} - 1) = 6\left(\frac{n}{2} - 1\right) = \mathcal{O}(n).$$

Exercise 2: Let I, J be index sets and $A \in \mathbb{R}^{I \times J}$. For $K \in \{I, J\}$ let T_K be a cluster tree to the index set K with $\#\,\mathrm{S}(\rho) \neq 1$ for all $\rho \in T_K$. Let $T_{I\times J}$ a block cluster tree constructed from the cluster trees T_I, T_J and an admissibility condition (compare equation (5.1) in the lecture notes). Further let $k_\rho \in \mathbb{N}$ with $k_\rho \leq \#\rho$ for all $\rho \in T_I$ and for all $\rho \in T_J$. For all $\rho \in L(T_I)$ and for all $\rho \in L(T_J)$ it should hold that $k_\rho = \#\rho$.
We want to find a matrix $\hat{A} \in \mathcal{H}^2(L(T_{I\times J}), \{\mathcal{V}_\tau\}_{\tau \in T_I}, \{\mathcal{W}_\sigma\}_{\sigma \in T_J})$, such that $\|A - \hat{A}\|_F$ is as small as possible.
Think about how subspaces $\{\mathcal{V}_\tau\}_{\tau \in T_I}$, $\{\mathcal{W}_\sigma\}_{\sigma \in T_J}$ that fulfill the nesting principle as well as corresponding orthonormal bases can be constructed such that the above approximation is as accurate as possible. The subspaces should be constructed such that $\dim(\mathcal{V}_\tau) = k_\tau$ holds for all $\tau \in T_I$ and $\dim(\mathcal{W}_\sigma) = k_\sigma$ for all $\sigma \in T_J$.
Solution hints:

In the following we first determine an orthonormal system U (i. e. a matrix (or a family of vectors) whose columns all have norm 1 and are pairwise orthogonal) and afterwards set $\mathcal{U} := \mathrm{Im}(U)$. Then U is an orthonormal basis of $\mathcal{U}$ by definition.
First we construct subspaces $\{\mathcal{V}_\tau\}_{\tau \in T_I}$ which fulfill the nesting principle by constructing orthonormal systems $V_\tau = (v_1^\tau, \ldots, v_{k_\tau}^\tau) \in \mathbb{R}^{\#\tau \times k_\tau}$ for all $\tau \in T_I$ and then set $\mathcal{V}_\tau := \mathrm{Im}(V_\tau)$. Then V_τ is an orthonormal basis of $\mathcal{V}_\tau \subset \mathbb{R}^{\#\tau}$ with $\dim(\mathcal{V}_\tau) = k_\tau$. For the construction of the orthonormal systems V_τ we proceed as follows:

1.) For all $\tau \in L(T_I)$ there holds $k_\tau = \#\tau$ and hence $V_\tau = I_\tau \in \mathbb{R}^{\#\tau \times \#\tau}$ is an orthonormal system.

2.) If $\mathrm{depth}(T_I) = 0$ hols, we are done. Otherwise we construct an orthonormal system $V_\tau = (v_1^\tau, \ldots, v_{k_\tau}^\tau) \in \mathbb{R}^{\#\tau \times k_\tau}$ for all $\tau \in T_I^{(\mathrm{depth}(T_I)-1)}$ (this set of all clusters of a level is defined on sheet 10 in exercise 2).

   Because of $\tau \in T_I^{(\mathrm{depth}(T_I)-1)}$ it holds that $\mathrm{S}(\tau) \subset L(T_I)$ and therefore the nesting principle $\mathrm{Im}(V_\tau)|_{\tau_i} \subset \mathcal{V}_{\tau_i} = \mathrm{Im}(I_{\tau_i}) = \mathbb{R}^{\#\tau_i}$ is fulfilled for all $\tau_i \in \mathrm{S}(\tau)$ and for all $V_\tau \in \mathbb{R}^{\#\tau \times k_\tau}$.

Hence we only have to answer the question which orthonormal system $V_\tau \in \mathbb{R}^{\#\tau \times k_\tau}$ leads to a best approximation $\|A - \hat{A}\|_F$.

We define the influence range

$$E(\tau) := (\tau \times J) \setminus \bigcup_{\substack{\tau_i \in \mathrm{S}(\tau), \\ \tau_i \times \sigma \in T_{I \times J}}} \tau_i \times \sigma$$

and let $J(\tau) \subset J$ be the uniquely determined index set given by $E(\tau) = \tau \times J(\tau)$. Further we set $J_\tau := \#J(\tau)$. Subsequently we compute an SVD of $B := A|_{E(\tau)} \in \mathbb{R}^{\#\tau \times J_\tau}$ ($B = V\Sigma W^T$) and set $V_\tau := V_{k_\tau}$ (where V_{k_τ} is the matrix, which consists of the first k_τ columns of V, and it holds $T^{\mathcal{R}}_{k_\tau \leftarrow \mathrm{rank}(B)}(B) = V_{k_\tau}\Sigma_{k_\tau}W^T_{k_\tau}$ (compare compressed SVD)).

Then we apply the update $A|_{E(\tau)} := V_\tau V_\tau^T A|_{E(\tau)}$ (compare exercise 1b)) and set $j := 1$.

3.) If $\mathrm{depth}(T_I) = j$, we are done. Otherwise we construct an orthonormal system $V_\tau = (v_1^\tau, \dots, v_{k_\tau}^\tau) \in \mathbb{R}^{\#\tau \times k_\tau}$ for all $\tau \in T_I^{(\mathrm{depth}(T_I)-1-j)}$.

Let $\mathrm{S}(\tau) = \{\tau_1, \dots, \tau_{\#\mathrm{S}(\tau)}\}$. Then $\#\tau = \sum_{i=1}^{\#\mathrm{S}(\tau)} \#\tau_i$. We define

$$\hat{V}_\tau := \begin{bmatrix} V_{\tau_1} & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & V_{\tau_{\#\mathrm{S}(\tau)}} \end{bmatrix} \in \mathbb{R}^{\#\tau \times \sum_{i=1}^{\#\mathrm{S}(\tau)} k_{\tau_i}}$$

and compute $B := \hat{V}_\tau \hat{V}_\tau^T A|_{E(\tau)} \in \mathbb{R}^{\#\tau \times J_\tau}$. Then we determine an SVD $B = V\Sigma W^T$ of B and set (analogous to step 2.)) $V_\tau := V_{k_\tau}$.

Then we compute the update $A|_{E(\tau)} := V_\tau V_\tau^T A|_{E(\tau)}$ and repeat step 3.) for $j := j + 1$.

To construct the subspaces $\{\mathcal{W}_\sigma\}_{\sigma \in T_J}$ we proceed analogously to the construction of the subspaces $\{\mathcal{V}_\tau\}_{\tau \in T_I}$, but now we do not start with the original matrix A but instead with the trasposed matrix of A which we obtained after applying all the previous updates above.
Remark: In steps 2.) and 3.) we can speed up the computation of V_τ. To this end compute an SVD of $BB^T \in \mathbb{R}^{\#\tau \times \#\tau}$ instead of an SVD of $B \in \mathbb{R}^{\#\tau \times J_\tau}$. Here we use that the same $V \in \mathbb{R}^{\#\tau \times \#\tau}$ belongs to an SVD of B as well as to an SVD of BB^T, and in applications it usually holds that $\#\tau \leq J_\tau$ (for $\tau \in T_I$ with $\mathrm{level}(\tau)$ sufficiently large it often holds $\#\tau \ll J_\tau$). In step 3.) we can reduce the complexity further because from the first exercise on this exercise sheet we have $\mathrm{Im}(B) \subset \mathrm{Im}(\hat{V}_\tau)$, i. e. in particular every column of B is contained in $\mathrm{Im}(\hat{V}_\tau)$. Therefore a matrix $Z_\tau \in \mathbb{R}^{J_\tau \times \sum_{i=1}^{\#\mathrm{S}(\tau)} k_{\tau_i}}$ exists with $B = \hat{V}_\tau Z_\tau^T$. If an SVD $Z_\tau^T Z_\tau = \tilde{V}\tilde{\Sigma}\tilde{W}^T$ of $Z_\tau^T Z_\tau$ is computed, we use

$$BB^T = \hat{V}_\tau Z_\tau^T Z_\tau \hat{V}_\tau^T = \hat{V}_\tau \tilde{V}\tilde{\Sigma}\tilde{W}^T \hat{V}_\tau^T = V\tilde{\Sigma}V^T$$

to show that it is an SVD of BB^T (where $V := \hat{V}_\tau \tilde{V}$ is an orthogonal matrix and belongs to an SVD of B). So instead of an SVD of $BB^T \in \mathbb{R}^{\#\tau \times \#\tau}$ (or an SVD of $B \in \mathbb{R}^{\#\tau \times J_\tau}$) an SVD of $Z_\tau^T Z_\tau \in \mathbb{R}^{\sum_{i=1}^{\#\mathrm{S}(\tau)} k_{\tau_i} \times \sum_{i=1}^{\#\mathrm{S}(\tau)} k_{\tau_i}}$ can be computed, where $\sum_{i=1}^{\#\mathrm{S}(\tau)} k_{\tau_i} \leq \#\tau$ holds (for $\tau \notin L(T_I)$ equality often does not hold here).

Exercises (in class):

Exercise 3: In *clusterbasis.h* the following struct `clusterbasis` is defined.

```
typedef struct _clusterbasis clusterbasis;
typedef clusterbasis *pclusterbasis;
typedef const clusterbasis *pcclusterbasis;
struct _clusterbasis {
  pccluster t;
  double **T;
  double *V;
  double *xt;
  double *yt;
  int k;
  int kt;
  int n;
  int sons;
  pclusterbasis *son;
};
```

In `t`, the corresponding cluster τ is stored whose size $\#\tau$ is given by `n`, and if τ is a leaf, an `n` $\times$ `k` matrix will be saved in `V` whose columns form a basis of this leaf (in the lecture notes this matrix is called V_τ (τ is the row index set) and respectively W_σ (σ is the column index set)). In `sons` the number $\#\,\mathrm{S}(\tau)$ of successors of the cluster τ is saved where the `clusterbasis` of the ith successor τ_i can be found in `son[i]` and the corresponding `kt` $\times$ `kt` transfer matrix $T_{\tau\leftarrow\tau_i}$ for `son[i]` is given by `T[i]`. `k` stands for the maximum number of columns of `V` and `kt` gives the actually used number (holds analogously for the sizes of the transfer matrices). Further it holds `xt` $= \hat{x}_\tau$ and `yt` $= \hat{y}_\tau$ (with the notation used for the forward- and the backward transformation).

Further the following struct `uniformmatrix` is defined in *uniformmatrix.h*.

```
typedef struct _uniformmatrix uniformmatrix;
typedef uniformmatrix *puniformmatrix;
typedef const uniformmatrix *pcuniformmatrix;
struct _uniformmatrix {
  pclusterbasis row;
  pclusterbasis col;
  int rows;
  int cols;
  int kr;
  int kc;
  int ktr;
  int ktc;
  double *S;
};
```

In `row` and `col` the corresponding `clusterbasis` of the clusters is saved. By `rows` and `cols` the number of rows and columns of these clusters are given. In `S`, a `kr` $\times$ `kc` matrix is stored

which is called K in the lecture notes, and the actually used size of this matrix is `ktr` $\times$ `ktc`.

The `supermatrix`-format which was introduced on sheet 4 is extended by the following three components such that also $\mathcal{H}^2$-matrices can be stored in this format.

```
pclusterbasis row_cluster_basis;
pclusterbasis col_cluster_basis;
puniformmatrix u;
```

For an $\mathcal{H}^2$-matrix, each cluster is assigned a `clusterbasis` for the rows and for the columns. Further the admissible leaves of a $\mathcal{H}^2$-matrix are assigned a `uniformmatrix` instead of an `rkmatrix`. If no $\mathcal{H}^2$-matrix is present, there holds
`row_cluster_basis` = NULL, `col_cluster_basis` = NULL and `u` = NULL.
Due to the extension of the `supermatrix`-format the functions *new_supermatrix*, *getsize_supermatrix*, *del_supermatrix* and *addeval_supermatrix* in *supermatrix.c* were modified such that they can also be used for $\mathcal{H}^2$-matrices.
Implement the following functions in *clusterbasis.c*:

- `void forward_clusterbasis(pclusterbasis cb, const double *x);`

  for the computation of the forward transformation and

- `void backward_clusterbasis(pcclusterbasis cb, double *y);`

  for the computation of the backward transformation.

Implement the following functions in *uniformmatrix.c*:

- `void convertrk2_uniformmatrix(pcrkmatrix r, puniformmatrix u);`

  to convert a matrix saved in the `rkmatrix`-format to the `uniformmatrix`-format and

- `void addeval_uniformmatrix(puniformmatrix u, const double *v, double *w);`

  for the matrix-vector-multiplication in the form $\mathtt{w} := \mathtt{w} + \mathtt{u} * \mathtt{v} = \mathtt{w} + \mathtt{V} * \mathtt{S} * \mathtt{W}^{\mathtt{T}} * \mathtt{v}$ (this function is used in the modified *addeval_supermatrix* function for admissible leaves of an $\mathcal{H}^2$-matrix).

Test your functions with the help of *example12.c*.

Note: *example12.c* corresponds largely to the example programm *example11.c*, where among other things the call to the function

```
fill_kernelaca_supermatrix(s, d, rank, nodes_x, test_function_log, 0, 0, nd);
```

was replaced by the call to the functions

```
cb_root = create_aca_clusterbasis(ct->root, d, rank, delta, nodes_x,
                                  test_function_log, nd);
fill_kernelaca_h2_supermatrix(s, cb_root, cb_root, d, rank, delta, nodes_x,
                              test_function_log, 0, 0, nd);
```

The latter are already implemented in *clusterbasis.c* and *supermatrix.c*. There a `clusterbasis` is computed firstly, which uses rows as well as columns. Afterwards for each admissible leaf an `rkmatrix` is determined, which is then converted to an `uniformmatrix` by the function *convertrk2_uniformmatrix*.
Solution hints: